

LAPORAN JAWABAN TEORI & IMPLEMENTASI

Ujian Akhir Semester (UAS) - Sistem Terdistribusi
Tema: Pub-Sub Log Aggregator Terdistribusi dengan Idempotency,
Deduplication, & Transaksi

Nama Mahasiswa : Nurfauzan Gymnastiar

NIM : 11231073

Mata Kuliah : Sistem Paralel dan Terdistribusi

Program Studi : Teknik Informatika / Sistem Informasi

Tanggal Pengumpulan : 16 Juni 2026

A. Bagian Teori

T1 (Bab 1): Karakteristik Sistem Terdistribusi dan Trade-off Desain Pub-Sub Aggregator

Sistem terdistribusi memiliki tiga karakteristik utama, yaitu konkurensi proses (*concurrency*), ketiadaan clock global (*no global clock*), dan kegagalan independen (*independent failures*) (Coulouris dkk., 2012). Karakteristik ini memicu tantangan signifikan dalam merancang Log Aggregator berbasis Publish-Subscribe (Pub-Sub).

Desain Pub-Sub Aggregator dalam tugas ini melibatkan trade-off antara:

1. **Asinkronitas vs. Konsistensi Waktu Nyata:** Pub-Sub memberikan skalabilitas tinggi melalui *loose coupling* spasial dan temporal. Namun, terjadi delay pengiriman antara saat event diterbitkan di API aggregator hingga diproses oleh consumer. Hal ini membatasi kegunaan sistem untuk analisis waktu nyata yang membutuhkan latensi di bawah milidetik.
2. **Kompleksitas Penyimpanan vs. Latensi Enqueue:** Memasukkan event log langsung ke PostgreSQL akan membatasi throughput API karena keterbatasan I/O database. Oleh karena itu, antrean antara Redis digunakan untuk meng-enqueue pesan dengan cepat (latensi rendah). Namun, ini menciptakan overhead pemeliharaan broker tambahan dan risiko kehilangan data jika Redis crash sebelum data ditulis ke penyimpanan persisten.
3. **Ketahanan Kegagalan vs. Konsumsi Sumber Daya:** Untuk mencegah kehilangan pesan akibat kegagalan independen, sistem harus menggunakan mekanisme *acknowledgment* dan *retry*, yang konsekuensinya memicu pengiriman pesan duplikat (*at-least-once delivery*).

Dalam implementasi kami, trade-off ini diselesaikan dengan menggunakan Redis sebagai broker memori cepat untuk meminimalkan latensi respons API, dikombinasikan dengan PostgreSQL persisten untuk menjamin konsistensi data akhir (*eventual consistency*) lewat mekanisme deduplikasi.

T2 (Bab 2): Kapan Memilih Arsitektur Publish-Subscribe dibanding Client-Server? Alasan Teknis

Arsitektur Client-Server tradisional didasarkan pada komunikasi sinkron *request-reply* yang memiliki ketergantungan erat (*tight coupling*) dalam ruang (klien harus mengetahui alamat server) dan waktu (klien dan server harus aktif bersamaan) (Coulouris dkk., 2012). Arsitektur Publish-Subscribe (Pub-Sub) dipilih untuk log aggregator ini karena alasan teknis berikut:

1. **Decoupling Ruang (Space Decoupling):** Publisher tidak perlu mengetahui berapa banyak consumer yang memproses log, atau di mana mereka berada. Hal ini memungkinkan tim operasi menambah consumer baru (misalnya layanan keamanan atau audit) tanpa mengubah kode atau konfigurasi di sisi publisher.
2. **Decoupling Waktu (Time Decoupling):** Log dapat dikirim oleh publisher meskipun database PostgreSQL sedang mati atau mengalami degradasi performa. Event akan tersimpan dengan aman di broker Redis dan diproses oleh consumer ketika database kembali normal.
3. **Decoupling Sinkronisasi (Synchronization Decoupling):** Pengiriman log tidak memblokir aktivitas utama pada publisher. Publisher mengirim log secara asinkron tanpa menunggu database selesai menulis data, menghindari *bottleneck* performa pada sistem utama.

Sistem *request-reply* konvensional akan membebani aplikasi utama setiap kali database log mengalami kemacetan, sedangkan Pub-Sub mengisolasi beban tersebut pada broker perantara (Redis) sehingga menjaga ketersediaan sistem secara keseluruhan.

T3 (Bab 3): At-Least-Once vs. Exactly-Once Delivery; Peran Idempotent Consumer

Dalam sistem terdistribusi, komunikasi jaringan tidak pernah sempurna. Kegagalan tautan komunikasi, latensi tinggi, atau jatuhnya node penerima dapat menyebabkan hilangnya paket pengakuan (*acknowledgment*) (Coulouris dkk., 2012).

1. **At-Least-Once Delivery:** Menjamin bahwa setiap pesan dikirim sekurang-kurangnya satu kali ke penerima. Jika pengirim tidak menerima konfirmasi dalam waktu tertentu, ia akan mengirim ulang pesan tersebut. Protokol ini mudah diimplementasikan tetapi memiliki konsekuensi log duplikat akibat pengiriman ulang pesan yang sebenarnya sudah diterima namun konfirmasinya hilang.
2. **Exactly-Once Delivery:** Menjamin bahwa setiap pesan diproses tepat satu kali. Secara teori, mencapai *exactly-once* murni pada lapisan transportasi jaringan sangat mahal dan hampir tidak mungkin tanpa koordinasi terdistribusi tingkat tinggi (seperti transaksi dua fase/2PC).
3. **Peran Idempotent Consumer:** Karena *exactly-once* sulit dicapai pada lapisan jaringan, arsitektur terdistribusi modern menerapkan pengiriman *at-least-once* di lapisan transportasi, dikombinasikan dengan konsumen idempoten (*idempotent consumer*) di lapisan aplikasi.

Konsumen idempoten memastikan bahwa jika pesan yang sama diproses berulang kali, efek sampingnya terhadap keadaan sistem (state) tetap sama dengan proses tunggal. Dalam rancangan kami, consumer menggunakan tabel *processed_events* PostgreSQL dengan constraint unik pada pasangan (*topic*, *event_id*) untuk memastikan pesan duplikat diabaikan dengan aman secara atomik tanpa merusak database.

T4 (Bab 4): Skema Penamaan Topic dan Event_ID untuk Deduplikasi

Penamaan dan pengidentifikasian entitas secara unik merupakan salah satu pilar utama komunikasi sistem terdistribusi (Coulouris dkk., 2012). Untuk mendukung deduplikasi yang kuat, sistem kami menerapkan skema penamaan terstruktur:

1. **Skema Topic:** Menggunakan format hierarkis berbasis titik (mis. *system.auth*, *payment.checkout*). Penamaan ini membagi ruang nama (*namespace*) menjadi partisi-partisi logis. Secara teknis, pemisahan topik ini memungkinkan sistem melakukan optimasi pengindeksan di tingkat database dan mempermudah perutean pesan.
2. **Event ID:** Identifikasi event harus bersifat unik secara global (*globally unique*) dan tahan terhadap tabrakan (*collision-resistant*). Kami menggunakan format *evt_[UUIDv4]*. UUIDv4 menghasilkan nilai acak 128-bit yang secara matematis menjamin tidak adanya tabrakan ID meskipun dihasilkan secara paralel oleh jutaan instansi publisher yang berbeda tanpa koordinasi terpusat.
3. **Kombinasi Kunci Unik:** Deduplikasi dilakukan menggunakan indeks unik gabungan (*composite unique index*) di PostgreSQL pada kolom (*topic*, *event_id*). Penggunaan kombinasi ini mencegah skenario di mana sistem yang berbeda secara tidak sengaja menghasilkan ID yang sama pada topik yang berbeda, serta mengoptimalkan pencarian database (*index lookup*) saat mendeteksi duplikat.

T5 (Bab 5): Ordering Praktis (Timestamp + Monotonic Counter); Batasan dan Dampaknya

Menurut Leslie Lamport, tidak ada *physical clock* yang benar-benar sinkron di seluruh mesin dalam sistem terdistribusi karena fenomena *clock drift* (Coulouris dkk., 2012). Oleh karena itu, mengurutkan log hanya berdasarkan waktu fisik host generator dapat menghasilkan urutan yang salah (misalnya, event B didefinisikan terjadi sebelum event A karena clock mesin B tertinggal).

Strategi Praktis Urutan Log: Untuk mengatasi keterbatasan ini, sistem kami menggabungkan timestamp fisik ISO 8601 dengan generator counter monotonik di sisi pengirim. Urutan akhir dievaluasi berdasarkan pasangan tuple (*timestamp*, *counter*).

Batasan dan Dampaknya:

1. **Clock Drift:** Jika sebuah host mengalami penyimpangan waktu yang ekstrem, log dari host tersebut dapat terdaftar di masa depan atau masa lalu yang jauh di database pusat, meskipun urutan internal log dari host yang sama tetap terjaga oleh counter monotonik.
2. **Urutan Parsial vs Total:** Pengurutan ini hanya menjamin *partial ordering* (kejadian berurutan dalam satu sumber pengirim), bukan *total ordering* (urutan mutlak dari semua pengirim yang berbeda).
3. **Toleransi Out-of-Order:** Di sisi database aggregator, log disimpan menggunakan waktu penerimaan aslinya. Karena pengiriman asinkron melalui antrean Redis, event mungkin tiba tidak berurutan (*out-of-order*). Namun, sistem tidak menolak data tersebut, melainkan menyimpannya sebagaimana adanya dan menyerahkan urutan visualisasi akhir kepada query SQL konsumen dengan klausa *ORDER BY timestamp ASC*.

T6 (Bab 6): Failure Modes dan Mitigasi dalam Sistem Terdistribusi

Sistem terdistribusi rentan terhadap berbagai jenis kegagalan. Coulouris dkk. (2012) mengklasifikasikan kegagalan menjadi kegagalan proses/node (*crash-stop*, *crash-recovery*) dan kegagalan saluran komunikasi (*arbitrary/omission failures*).

Berikut adalah tabel failure modes dan mitigasi yang kami terapkan:

Jenis Kegagalan	Efek / Failure Mode	Mitigasi dalam Rancangan Kami
Crash pada Node Database	Consumer gagal menyimpan data, memicu data loss.	Antrean Redis menahan pesan sementara. Consumer mendeteksi kegagalan koneksi database, melakukan retry dengan exponential backoff, dan tidak melakukan acknowledgement (BLPOP diproses transaksional/transaksi diulang) hingga DB pulih.
Crash pada Node Consumer	Pesan yang sedang diproses hilang di tengah jalan.	Penggunaan antrean asinkron Redis. Jika consumer mati, koneksinya terputus, dan pesan tetap aman di Redis untuk ditarik oleh worker lain.
Kegagalan Saluran Jaringan	Paket terputus, memicu pengiriman ulang oleh publisher.	Deduplikasi berbasis PostgreSQL unique constraint memfilter log duplikat secara atomik di sisi consumer.
Crash-Recovery Container	Seluruh container dihapus dan dinyalakan ulang.	Data PostgreSQL disimpan pada Named Volumes Docker luar kontainer untuk mencegah hilangnya riwayat deduplikasi.

T7 (Bab 7): Eventual Consistency pada Aggregator; Peran Idempotency dan Deduplikasi

Model konsistensi pada log aggregator ini adalah konsistensi akhir (*eventual consistency*), di mana data replika atau status penyimpanan tidak langsung diperbarui secara instan pada saat event dikirim, tetapi dijamin akan konsisten di semua node setelah waktu tertentu jika tidak ada input baru yang masuk (Coulouris dkk., 2012).

Bagaimana Idempotensi dan Deduplikasi Menjamin Eventual Consistency:

1. **Toleransi Retry:** Jika publisher mengirim log dan gagal menerima HTTP status 202, publisher akan mengirim ulang log tersebut. Tanpa deduplikasi, sistem akan mencatat log ganda, merusak metrik statistik.
2. **Deduplikasi Atomik:** Melalui operasi *INSERT ... ON CONFLICT DO NOTHING*, database PostgreSQL bertindak sebagai *single source of truth* yang menyaring duplikasi pesan.
3. **Konsistensi Metrik:** Perhitungan statistik diperbarui di dalam transaksi yang sama dengan penyisipan event log. Jika database mendeteksi data unik, nilai *unique_processed* bertambah. Jika terdeteksi duplikat, nilai *duplicate_dropped* bertambah.

Melalui kombinasi ini, meskipun terdapat pengiriman ulang log berkali-kali oleh ratusan worker, keadaan akhir metrik statistik database pada akhirnya akan selalu sama dan akurat, memenuhi prinsip dasar eventual consistency.

T8 (Bab 8): Desain Transaksi: ACID, Isolation Level, dan Strategi Menghindari Lost-Update

Mencegah anomali konkurensi di bawah beban konkuren tinggi memerlukan jaminan transaksi ACID (Atomicity, Consistency, Isolation, Durability) (Coulouris dkk., 2012).

Dalam tugas ini, kita memproses pembaruan counter statistik secara bersamaan dari beberapa consumer worker paralel. Anomali utama yang harus dihindari adalah **Lost-Update**, di mana dua transaksi membaca nilai statistik yang sama secara bersamaan, meningkatkan nilainya, lalu menuliskannya kembali secara berurutan, menyebabkan satu peningkatan hilang.

Pilihan Isolation Level dan Trade-off:

1. READ COMMITTED (Pilihan Kami):

* **Mekanisme:** Setiap kueri hanya melihat data yang telah dideklarasikan selesai (*committed*) sebelum kueri dimulai.

* **Strategi Menghindari Lost-Update:** Kami menggunakan pembaruan atomik langsung di database: `UPDATE stats SET value = value + 1 WHERE key = 'unique_processed'`. Kueri ini secara implisit memperoleh kunci baris (*row lock*) eksklusif pada baris stats yang bersangkutan. Jika transaksi lain mencoba memperbarui baris yang sama, ia akan diblokir hingga transaksi pertama selesai. Nilai yang dibaca oleh transaksi kedua adalah nilai terupdate dari transaksi pertama.

* **Trade-off:** Kinerja sangat tinggi dengan latensi rendah karena tidak ada pembatalan transaksi akibat konflik isolasi.

2. SERIALIZABLE:

* **Mekanisme:** Menjamin hasil eksekusi paralel sama persis dengan eksekusi sekuensial.

* **Trade-off:** Jika dua transaksi mencoba memperbarui baris stats secara bersamaan, transaksi kedua akan segera gagal dengan kesalahan serialisasi (`SQLState 40001: could not serialize access`). Ini membutuhkan mekanisme penanganan kesalahan tingkat aplikasi berupa pengulangan transaksi secara agresif dengan backoff, meningkatkan latensi dan menurunkan throughput keseluruhan sistem.

Oleh karena itu, kombinasi **READ COMMITTED dengan Row-level Locking** dinilai paling optimal dan aman untuk kebutuhan performa log aggregator ini.

T9 (Bab 9): Kontrol Konkurensi: Locking vs. Unique Constraints vs. Idempotent Upsert

Kontrol konkurensi terdistribusi dapat dikelola melalui pendekatan pesimis (*pessimistic locking*) atau optimis (*optimistic locking/no-locking*) (Coulouris dkk., 2012).

1. **Locking Pesimis (Pessimistic Locking):** Menggunakan perintah seperti *SELECT ... FOR UPDATE*. Consumer mengunci baris di database sebelum memeriksa apakah event tersebut sudah ada. Pendekatan ini mencegah modifikasi oleh pihak lain, tetapi menciptakan overhead *deadlock* dan menurunkan throughput sistem secara drastis di bawah beban paralel.
2. **Unique Constraints (Lockless Optimistic Approach):** Memanfaatkan indeks database unik pada (*topic, event_id*). Pendekatan ini tidak menahan kunci sebelum operasi, melainkan langsung mencoba menulis data dengan asumsi tidak terjadi konflik. Jika konflik terjadi, database menolaknya di tingkat mesin penyimpanan.
3. **Idempotent Upsert (Pilihan Kami):** Kami menerapkan klausa SQL standard *ON CONFLICT (topic, event_id) DO NOTHING*. Operasi ini sangat efisien karena dikerjakan secara atomik dalam satu langkah instruksi penulisan database. Mesin database secara internal mengunci baris indeks hanya dalam durasi microsecond untuk validasi keunikan, serta mencegah pembatalan transaksi.

Mekanisme ini memungkinkan beberapa consumer worker memproses pesan duplikat secara paralel tanpa saling memblokir satu sama lain, memaksimalkan konkurensi sistem paralel.

T10 (Bab 10-13): Orkestrasi Compose, Keamanan Jaringan Lokal, Persistensi Volume, dan Observability

Mengoperasikan sistem terdistribusi di lingkungan produksi memerlukan dukungan infrastruktur penunjang berupa orkestrasi kontainer, isolasi jaringan, manajemen penyimpanan, dan pemantauan sistem (Coulouris dkk., 2012).

1. **Orkestrasi Docker Compose:** Mengelola siklus hidup multi-layanan (API, consumer, Redis, PostgreSQL) dalam satu kesatuan arsitektur deklaratif. Melalui konfigurasi *depends_on* dengan *condition: service_healthy*, Docker Compose memastikan urutan startup yang benar.
2. **Keamanan Jaringan Lokal:** Semua layanan berjalan pada jaringan internal bridge Compose (*internal-net*). Layanan database dan broker ****tidak mengekspos port**** ke host luar. Hanya port 8080 pada API yang diekspos keluar untuk diakses oleh klien luar. Hal ini mengurangi bidang serang keamanan sistem.
3. **Persistensi Data (Volumes):** Menggunakan *named volumes* (*pg_data* dan *broker_data*). Volume ini memetakan folder di dalam kontainer ke penyimpanan host fisik. Jika kontainer PostgreSQL dihancurkan dan dibuat ulang, data log unik yang telah diproses dan status metrik statistik tetap utuh dan aman.
4. **Observability:** Sistem memantau kondisi operasional melalui *Readiness/Liveness Probe* (endpoint `/health/readiness` secara dinamis memeriksa konektivitas database dan redis) dan *Metrik Terpusat* (endpoint `/stats` merangkum jumlah log masuk, duplikat yang dibuang, uptime secara transaksional).

B. Analisis Alur Kerja Step-by-Step & Visualisasi Data

Bagian ini memaparkan alur kerja sistem log aggregator secara kronologis beserta transformasi skema visualisasi data pada setiap tahapannya.

Langkah 1: Pembangkitan Event Log (Sisi Publisher)

Simulator membangkitkan log telemetry menggunakan generator data terdistribusi. Untuk menguji toleransi duplikasi (at-least-once delivery), 30% dari event log yang sama dikirim kembali dengan timestamp pengiriman ulang baru.

```
{
  "topic": "system.auth",
  "event_id": "evt_ab8c92f15e8b",
  "timestamp": "2026-06-16T16:34:41.123456Z",
  "source": "auth-service",
  "payload": {
    "request_id": "req_8ff3a21b",
    "execution_time_ms": 42.15,
    "status": "success"
  }
}
```

Langkah 2: Penerimaan & Validasi (Sisi Aggregator API)

FastAPI menerima muatan data di endpoint /publish. Model data divalidasi secara asinkron menggunakan Pydantic. API kemudian meningkatkan counter received secara transaksional di PostgreSQL, lalu serialisasi event ke string JSON dan memasukannya ke antrean Redis (event_queue) dengan perintah RPUSH.

Langkah 3 & 4: Penyaluran Asinkron & Penyimpanan Relasional (Sisi Consumer & DB)

Consumer worker menarik pesan dari Redis antrean menggunakan operasi BLPOP. Worker membuka transaksi relasional PostgreSQL dengan tingkat isolasi READ COMMITTED. Kueri INSERT ... ON CONFLICT DO NOTHING dipicu secara atomik menggunakan constraint unik pada kolom indeks UNIQUE(topic, event_id).

Visualisasi Skema Tabel processed_events:

Nama Kolom	Type Data	Kunci / Indeks	Deskripsi
id	SERIAL	PRIMARY KEY	ID sekuensial internal.
topic	VARCHAR(255)	UNIQUE INDEX (1)	Nama kategori topik log.
event_id	VARCHAR(255)	UNIQUE INDEX (2)	Pengenal unik global event.

timestamp	TIMESTAMPTZ	-	Waktu fisik event terjadi.
source	VARCHAR(255)	-	Layanan asal pengirim log.
payload	JSONB	-	Payload detail log terindeks.

Langkah 5: Pembaruan Metrik Konsistensi Akhir (Sisi PostgreSQL Stats)

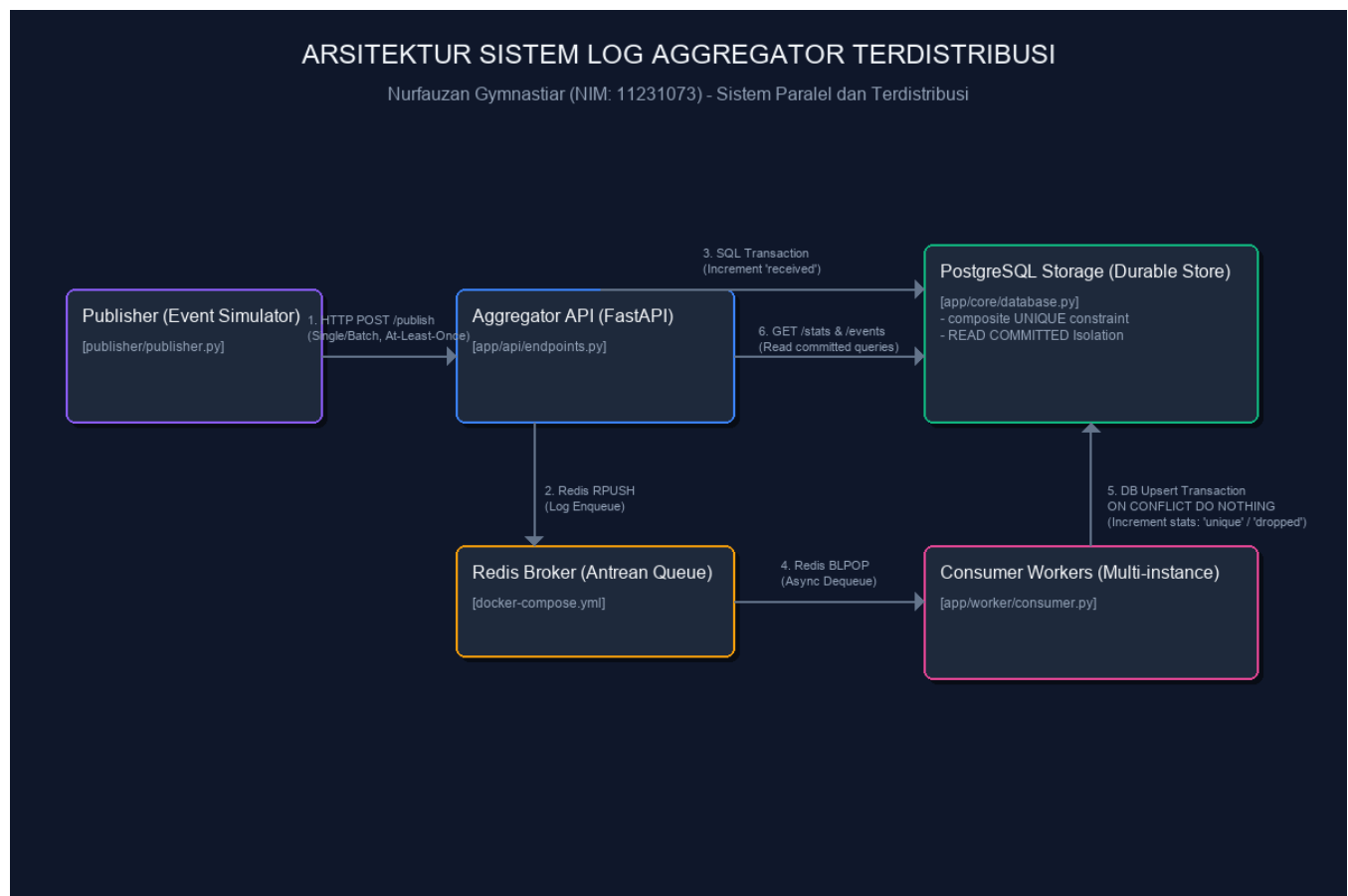
Berdasarkan hasil eksekusi kueri pada Langkah 4, database mengembalikan row ID jika baris berhasil dimasukkan (log unik baru), atau mengembalikan kosong jika terjadi tabrakan indeks (log duplikat).

Worker memicu kueri UPDATE stats SET value = value + 1 untuk baris stats yang bersangkutan di bawah row-level locking.

Visualisasi Skema Tabel stats:

key (VARCHAR - PK)	value (BIGINT)	Status Kunci Baris (Row-Locking)
received	20000	Dikunci sementara saat API menerima log
unique_processed	14000	Dikunci sementara saat Consumer menulis log unik
duplicate_dropped	6000	Dikunci sementara saat Consumer mendeteksi duplikat

C. Diagram Visualisasi Arsitektur Terdistribusi



Daftar Pustaka

Coulouris, G., Dollimore, J., Kindberg, T., & Blair, G. (2012). *Distributed Systems: Concepts and Design* (5th ed.). Addison-Wesley.

Decandia, G., Hastorun, D., Jampani, M., Kakulapati, G., Lakshman, A., Pilchin, A., Sivasubramanian, S., Vosshall, P., & Vogels, W. (2007). Dynamo: Amazon's Highly Available Key-value Store. *ACM SIGOPS Operating Systems Review*, 41(6), 205-220.

Kreps, J., Narkhede, N., & Rao, J. (2011). Kafka: a Distributed Messaging System for Log Processing. *Proceedings of the NetDB*, 1-7.

Stonebraker, M., Madden, S., & Abadi, D. J. (2007). The End of an Architectural Era (It's Time for a Complete Rewrite). *Proceedings of the 33rd International Conference on Very Large Data Bases (VLDB)*, 1150-1160.